Day 1-2

LLM + Container

From Fundamentals to Hands-on Deployment

1 LLM Fundamentals

2 Ecosystem & Tools

3 Container Basics

W Workshop

Agenda

1

LLM Fundamentals

- LLM คืออะไร? Decoder-only
- Model size: 7B / 13B / 70B
- Training / Fine-tuning / Inference
- Precision: FP16 / INT8 / INT4

2

LLM Ecosystem & Tools

- Hugging Face / Transformers / vLLM
- TGI / Ollama / bitsandbytes / LoRA
- เลือก tool ให้เหมาะกับงาน

3

Container Basics

- Container vs VM
- Docker: build, run, push
- Singularity บน HPC / SLURM
- NGC: GPU-optimized containers
- GPU + Container

W

Workshop: Tech Stack Advisor

- Clone → conda → Train
- Dockerize → Build → Push
- Singularity บน HPC
- Deploy บน Hugging Face Spaces

1

LLM Fundamentals

เข้าใจ LLM จาก System View

LLM คืออะไร?

🧠 **Model ขนาดใหญ่** โปรแกรมที่เรียนรู้ภาษามนุษย์จากข้อความมหาศาล เหมือนคนอ่านหนังสือทุกเล่มในโลก

📊 **Parameter** ตัวเลขนับพันล้านตัวที่เก็บ 'ความรู้' ของ model เช่น 8B = 8 พันล้าน parameters

💬 **Inference** การนำ model ที่ train แล้วมาตอบคำถาม — สิ่งที่ผู้ใช้ทั่วไปทำทุกวัน

🏋 **Training / Fine-tuning** การสอน model ให้รู้เรื่องเฉพาะทาง เช่น สอนให้รู้เรื่องกฎหมายไทย หรือเวชระเบียน

ทำไมต้องใช้ GPU? ไม่ใช้ CPU ได้ไหม?

🖥️ **CPU — ทำทีละงาน** เหมือนพ่อครัวมีฝีมือ 1 คน ทำอาหารได้ดีมาก แต่ทำได้ทีละจาน

🎮 **GPU — ทำพร้อมกันหลายพันงาน** เหมือนมีผู้ช่วยพันคน แต่ละคนทำงานง่าย ๆ พร้อมกัน — เหมาะกับการคำนวณ matrix ของ LLM

📏 **ขนาด model → GPU ที่ต้องใช้** Typhoon 8B ≈ 16GB VRAM | Typhoon 70B ≈ 140GB VRAM

🖥️ **RSU Cluster มี** NVIDIA H200 (141GB) และ H100 (94GB) — รองรับ model ขนาดใหญ่ได้

Model Size — 7B / 13B / 70B หมายความว่าอะไร?

"B" = Billion parameters • parameters = weights ที่เก็บความรู้ของ model

	7B	13B	70B	7B INT4
VRAM:	~14 GB	~26 GB	~140 GB	~4-5 GB
Precision:	FP16	FP16	FP16	INT4 (Q)
Hardware:	RTX 3090 / A100 40GB	A100 40-80GB	H100×2 / A100×4	RTX 3060 / MIG 1g
Speed:	เร็ว	ปานกลาง	ช้า	เร็ว
Use case:	inference ทั่วไป, fine-tune ได้	คุณภาพสูง, summarize, code	research, GPT-4 class	ลด VRAM ด้วย quantization

🎯 **takeaway: LLM ข้อจำกัดหลัก = GPU Memory**

Token / Context / Training vs Inference

Token & Context

Token คืออะไร?

"Hello" → ["Hel", "lo"] • Thai 1 คำ ≈ 2-4 tokens

1 token ≈ 4 ตัวอักษรภาษาอังกฤษ

Context Length

จำนวน token สูงสุดที่รับได้ใน 1 request

Llama 3.1 / GPT-4o: 128K tokens • ยาวขึ้น = VRAM มากขึ้น

Precision (Quantization)

FP32 4 bytes/param → ช้า, แม่นที่สุด

FP16/BF16 2 bytes/param → balance – ใช้ทั่วไป

INT8 1 byte/param → เร็ว, คุณภาพ -5%

INT4 0.5 byte/param → เร็วมาก, VRAM น้อยที่สุด

Training vs Fine-tuning vs Inference

🏗️ Pre-training

เรียนจาก data หลายพัน billion tokens

GPU cluster หลายร้อยตัว หลายสัปดาห์

→ GPT, Llama, Typhoon

🎯 Fine-tuning

ปรับ model ให้เชี่ยวชาญงานเฉพาะ

GPU น้อยกว่า, LoRA/QLoRA popular

→ ใช้ dataset ของเราเอง

⚡ Inference

นำ model ที่ train แล้วมาใช้งาน

ต้องการ GPU น้อยกว่า training มาก

→ Deploy ใน container บน cluster

2

LLM Ecosystem & Tools

ภาพรวมเครื่องมือที่ใช้งานจริงในสาย HPC / Production

LLM Tool Ecosystem — เลือกให้ถูก

Tool	ประเภท	ใช้ทำอะไร
🤗 Hugging Face	Model Hub / Datasets	ดาวน์โหลด model, dataset, Spaces
🔧 Transformers	Python Library	API สำหรับ load & run model ทุกชนิด
⚡ vLLM	High-perf Inference	Throughput สูง, PagedAttention, production-grade
🚀 TGI	Production Server	Hugging Face's inference server, REST API
🦙 Ollama	Local LLM Runner	รัน model บน laptop ได้ง่าย (dev/test)
🧮 bitsandbytes	Quantization	INT8/INT4 quantize เพื่อลด VRAM
🎯 PEFT / LoRA	Fine-tuning	Fine-tune model ขนาดใหญ่ด้วย GPU น้อย
📦 Singularity	HPC Container	รัน container บน SLURM cluster (no root)

🎯 **takeaway: เลือก tool ผิด = ช้า / เปลือง VRAM / deploy ยาก**

SLURM คืออะไร? ทำไม HPC ต้องใช้?

📋 **Job Scheduler = ระบบจองคิว GPU** ผู้ใช้หลายคนต้องการ GPU พร้อมกัน SLURM จัดคิวให้ยุติธรรม ไม่ให้ใครผูกขาดทรัพยากร

📝 **Submit Job = ส่งใบจอง** เขียน script บอกว่า 'ต้องการ GPU กี่ใบ, RAM เท่าไร, ใช้เวลากี่ชั่วโมง' แล้วส่งให้ SLURM จัดการ

⏳ **Job Queue** ถ้า GPU ว่าง → รันทันที | ถ้า GPU ไม่ว่าง → รอคิว แล้วรันอัตโนมัติเมื่อมีทรัพยากร

📊 **RSU Cluster Resource Limits** CPU x cores | RAM x GB | GPU x ใบ (H200 MIG)

💡 *ไม่ต้องรอหน้าจอ! SLURM รันงานให้อัตโนมัติ แม้ปิด laptop แล้วก็ตาม*

SLURM vs Kubernetes — เลือกใช้เมื่อไร?

⚡ SLURM (Training)

- งานที่มีจุดเริ่มและจุดสิ้นสุด
- ต้องการ GPU มาก ในเวลาสั้น
- Fine-tuning, batch processing
- ไม่ต้องการ uptime ตลอดเวลา
- ตัวอย่าง: train model 4 ชั่วโมง

Kubernetes (Inference)

- Service ที่ต้องรันตลอด 24/7
- ผู้ใช้หลายคนเข้าพร้อมกัน
- API endpoint, Web UI
- ต้องการ high availability
- ตัวอย่าง: chatbot สำหรับนักศึกษา

Inference vs Training — ต่างกันอย่างไร?

🔍 Inference (การใช้งาน)

- ใช้ model ที่ train แล้วตอบคำถาม
- GPU 1-2 ใบก็พอ
- เร็ว — ตอบได้ภายในวินาที
- ทำบน Kubernetes (Workshop 3)
- ใช้งานได้ตลอด 24/7 เป็น Service

🏋 Training / Fine-tuning

- สอน model ให้รู้เรื่องใหม่ ๆ
- ต้องการ GPU หลายใบ, RAM มาก
- ช้า — อาจใช้เวลาหลายชั่วโมง/วัน
- ทำบน SLURM Cluster (Workshop 2)
- ผลลัพธ์คือไฟล์ model weight ใหม่

คำสั่ง SLURM ที่ใช้บ่อย

📤 ส่งและดูงาน

- sbatch job.sh → ส่งงาน
- squeue -u $USER → ดูงานของตัวเอง
- sinfo → ดูสถานะ cluster
- scancel <JOBID> → ยกเลิกงาน
- sacct -j <JOBID> → ดูประวัติงาน

⚡ รันแบบ Interactive

- srun --gres=gpu:1 -n1 --pty bash
- → ขอ GPU แล้ว เข้า shell ทันที
-
- srun --gres=gpu:1 nvidia-smi
- → รันคำสั่งเดียวแล้วออก

Docker คืออะไร? ทำไมต้องใช้?

📦 **Container = กล่องสำเร็จรูป** บรรจุ software, library, และ model ทุกอย่างไว้ในกล่องเดียว ไม่ต้องติดตั้งเอง

🔄 **ทำงานได้ทุกที่** เครื่อง laptop, VM, SLURM, Kubernetes — ใช้ image เดียวกันหมด

🧪 **ทดสอบได้ปลอดภัย** ลองผิดลองถูกใน container โดยไม่กระทบระบบอื่น

🚀 **Ollama บน Docker** Ollama = เครื่องมือ run LLM ที่ง่ายที่สุด รองรับ Typhoon, DeepSeek, Llama และอีกร้อยกว่า model

3

Container Basics

Container คืออะไร • Docker • Singularity • NGC • GPU

Container คืออะไร? — Container vs Virtual Machine

Virtual Machines (VM)

App 1

Bins / Libs

Guest OS (10s GB)

Hypervisor

Host OS

Server

⚠ Guest OS = 10s of GB overhead ต่อ VM

Containers (Docker / Singularity)

App 1	App 2	App 3
Bins / Libs	Bins / Libs	Bins / Libs

Docker Engine (shared kernel)

Host OS

Server

- ✅ ไม่มี Guest OS → เบาและเร็วกว่า VM มาก
- ✅ Package ทุกอย่างอยู่ใน image เดียว
- ✅ Build once, run anywhere
- ✅ Reproducible: ทุก node ได้ผลเหมือนกัน

Docker — สร้าง, รัน, และแจกจ่าย Container

Dockerfile
สูตร build image

→

Image
snapshot ของ env

→

Container
instance ที่รันอยู่

→

Registry
Docker Hub / NGC

Essential Commands

```bash
# Build image จาก Dockerfile
$ docker build -t myapp:v1 .

# Run container + map port
$ docker run -p 7860:7860 myapp:v1

# Run with GPU support
$ docker run --gpus all -p 8000:8000 myapp:v1

# Push to Docker Hub
$ docker push user/myapp:v1
```

Dockerfile ตัวอย่าง

```Dockerfile
FROM python:3.11-slim

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir \
    -r requirements.txt

COPY . .
EXPOSE 7860
CMD ["python", "app.py"]
```

⚠ Docker ต้องการ root → ใช้บน HPC ไม่ได้โดยตรง → ใช้ Singularity แทน

Singularity — Container Engine สำหรับ HPC

ทำไม HPC ต้องใช้ Singularity? • ไม่ต้อง root • ทำงานบน SLURM ได้ทันที • รองรับ MPI, InfiniBand, GPU

	Docker	Singularity
ต้อง root?	✅ ต้องการ	❌ ไม่ต้อง
บน SLURM?	⚠️ ต้อง Docker daemon	✅ ทำงานได้ทันที
GPU support	--gpus all	--nv flag
Image format	Layer-based	.sif (single file)
Convert from Docker	—	singularity pull docker://...

Singularity Workflow

bash

```bash
# Convert Docker image → SIF (ทำครั้งเดียว)
$ singularity pull myapp.sif docker://user/myapp:v1
# Run with GPU:  singularity exec --nv myapp.sif python app.py   # SLURM: #SBATCH --gres=gpu:1
```

NVIDIA GPU Cloud (NGC) — GPU-Optimized Containers

NGC คืออะไร?

NGC = hub ของ GPU-optimized software สำหรับ AI, ML, HPC ที่ NVIDIA ดูแลให้พร้อมใช้งาน
รับประกันว่า work กับ NVIDIA GPU ได้ทันที ไม่ต้องติดตั้ง CUDA, cuDNN เอง

NGC Container Registry

Docker/Singularity images สำหรับ PyTorch, TensorFlow, JAX, TensorRT, LLM inference engines
พร้อม CUDA stack ทั้งหมด

Pre-trained Models

Download model weights ที่ NVIDIA optimize แล้ว
สำหรับ NLP, CV, Speech, Recommendation

GPU-optimized Software

RAPIDS (GPU DataFrame), CUDA-X libraries
BioNeMo สำหรับ bioinformatics

https://catalog.ngc.nvidia.com/ • ใช้ Singularity pull nvcr.io/nvidia/<image> • ใช้ image PyTorch ใน Lab 4 ด้วย

GPU + Container — NVIDIA Container Toolkit

Architecture: GPU Container Stack

LLM App / ML Training

PyTorch / TensorFlow

CUDA Libraries (cuDNN, cuBLAS)

CUDA Runtime

Container (Docker / Singularity)

NVIDIA Container Toolkit

NVIDIA Driver (Host)

GPU Hardware (H200 / H100)

Running GPU Containers

Docker

```
# Docker + GPU
$ docker run --gpus all \
    -p 8000:8000 myapp:v1

# Verify GPU inside container
$ docker exec -it myapp nvidia-smi
```

Singularity

```
# Singularity + GPU
$ singularity exec --nv \
    myapp.sif python app.py

# NGC PyTorch container
$ singularity shell --nv \
    /cm/shared/sif/pytorch_24.02-py3.sif
```

💡 ที่ RSU: /cm/shared/sif/ มี PyTorch, TensorFlow NGC images พร้อมใช้งาน

Workshop

Tech Stack Advisor — ML App บน Docker & Singularity

Workshop: Tech Stack Advisor — ภาพรวม

🎯 ML App จริง (scikit-learn + Gradio) → ฝึก train → Dockerize → Deploy บน HPC ด้วย Singularity

Step 1

Setup & Train

- Fork + Clone repo
- conda activate pytorch-env
- python train.py
- (SLURM interactive job)
- python app.py → localhost:7860

Step 2

Dockerize App

- สร้าง Dockerfile
- docker build -t user/tsa:v2 .
- Test: docker run -p 7860:7860
- docker push to Docker Hub

Step 3

Singularity on HPC

- singularity pull .sif
- จาก Docker Hub
- singularity exec --nv .sif
- python app.py
- sbatch SLURM script

Step ★

Bonus: HF Spaces

- huggingface.co/spaces
- Create New Space
- SDK: Docker
- HF auto-build & host ฟรี

Workshop — Step 1: Setup & Train Model

📁 github.com/docker-aiml/tech-stack-advisor • scikit-learn + Gradio ML App

bash

```bash
# 1. Fork & Clone repo
$ git clone https://github.com/user/tech-stack-advisor.git
$ cd tech-stack-advisor

# 2. Activate conda env
$ source /cm/shared/apps/miniconda3/etc/profile.d/conda.sh
$ conda activate tsa

# 3. Train model (รันไม่กี่วินาที)
#    ทำบน SLURM interactive job
$ srun --gres=gpu:1 --mem=8G --pty bash
$ python train.py
   # สร้าง model.pkl + encoders.pkl

# 4. Test run locally
$ python app.py
   # → http://localhost:7860
```

App ทำอะไร?

Tech Stack Advisor รับ input:

→ Project Type (Web/API/ML/Real-time)

→ Team Size (slider)

→ Performance Need (Low/Med/High)

→ Experience Level (Beginner→Expert)

แล้ว Recommend Tech Stack
เช่น Django + SQLite / FastAPI + Redis

Project Structure

```
app.py  train.py  requirements.txt  model.pkl
encoders.pkl
```

Workshop — Step 2: Dockerize + Build + Push

สร้าง Dockerfile

Dockerfile

```
FROM python:3.11-slim

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .
EXPOSE 7860
CMD ["python", "app.py"]
```

Build, Test, Push

bash

```
# Build image
$ docker build -t <user>/tech-stack-advisor:v2 .

# Test run
$ docker run -p 7860:7860 <user>/tech-stack-advisor:v2

# Push to Docker Hub
$ docker push <user>/tech-stack-advisor:v2
```

💡 ทำความเข้าใจด้วย Manual Build (ก่อนใช้ Dockerfile)

bash — Manual approach (v1)

```
# 1. Start base container
$ docker run -idt --name dev \
    -p 7860:7860 python:3.11-slim bash

# 2. Copy source code in
$ docker cp . dev:/app

# 3. Install & run inside
$ docker exec -it dev bash
  cd app && pip install -r requirements.txt
  python app.py   # test at localhost:7860

# 4. Snapshot → image (v1)
$ docker container commit dev \
    <user>/tech-stack-advisor:v1
$ docker push <user>/tech-stack-advisor:v1
```

v1 = manual commit • v2 = Dockerfile
compare: docker image history user/tech-stack-advisor:v1 vs :v2

Workshop — Step 3: Singularity บน HPC

Step 3 — Pull Singularity Image from Docker Repository | Manual Build Singularity Image from Singularity Sandbox

bash

```bash
# 1. Convert Docker image → SIF
$ singularity pull tech-stack.sif \
    docker://<user>/tech-stack-advisor:v2

# 2. Run interactive with GPU
$ singularity exec --nv tech-stack.sif python app.py

# 3. Submit via SLURM
$ sbatch run_singularity.sh
```

run_singularity.sh

```bash
#!/bin/bash
#SBATCH --job-name=tsa    --gres=gpu:1
#SBATCH --mem=16G          --time=00:30:00

singularity exec --nv \
  tech-stack.sif python app.py
```

bash

```bash
# Run interactive with GPU
$ srun -c2 --gres=gpu:1 --mem=8G --pty bash

$ mkdir docker-aiml/
$ cd docker-aiml/
$ git clone https://github.com/docker-aiml/tech-stack-advisor
$ cd tech-stack-advisor/
$ apptainer build --fakeroot --sandbox tsa-dev docker://python:3.11-slim
$ apptainer shell --fakeroot tsa-dev

# Inside the container
Apptainer> pwd
Apptainer> cd docker-aiml/tech-stack-advisor/
Apptainer> ls
Apptainer> pip install -r requirements.txt
Apptainer> python train.py
Apptainer> python app.py
Apptainer> exit

$ apptainer build --fakeroot tsa-dev.sif tsa-dev/
$ apptainer exec --nv tsa-dev.sif python app.py
```

Workshop — Step 4: Bonus: HF Spaces

Get the Hugging Face token

Register Hugging Face account

- Go to Profile -> Access Token
- Create new token
- Select Fine-grained or Write
- Insert token name
- Click Create token
- Secure your token

🎁 Bonus: Hugging Face Spaces

1. ไปที่ huggingface.co/spaces
2. คลิก Create New Space
3. SDK: Docker
   Repository: Link GitHub repo ที่มี Dockerfile
4. HF จะ auto-build & deploy ให้อัตโนมัติ

```
https://huggingface.co/spaces/<username>/tech-stack-advisor
```

L

LLM App Workshop

Ollama + Typhoon บน Docker (CPU) และ SLURM + Singularity (GPU)

Docker Ollama บน rsu-training (CPU mode)

ทำไมใช้ CPU mode?

- rsu-training ไม่มี Docker GPU access
- ใช้ --runtime=runc แทน
- เหมาะสำหรับ demo concept + tinyllama
- Typhoon 8B ช้ามากบน CPU (ไม่แนะนำ)

Dynamic Port — ไม่ชนกันระหว่าง user

```
MY_PORT=$((11400 + $(id -u) % 1000))
→ UID 1043 = port 11443
→ UID 1006 = port 11406
→ แต่ละ user ได้ port ไม่ซ้ำกัน
```

bash — Docker Ollama (CPU, Dynamic Port)

```bash
# 1. คำนวณ port ของตัวเอง
MY_PORT=$((11400 + $(id -u) % 1000)) && echo "My port: $MY_PORT"

# 2. Run Ollama container (ไม่ใช้ GPU)
docker run -d --runtime=runc --name ollama-$USER \
  -p ${MY_PORT}:11434 -v $HOME/ollama_models:/root/.ollama ollama/ollama

# 3. Pull tinyllama + Chat
docker exec -it ollama-$USER ollama pull tinyllama
docker exec -it ollama-$USER ollama run tinyllama "What is Docker?"

# 4. REST API test
curl http://localhost:${MY_PORT}/api/tags
```

💡 ใช้ tinyllama สำหรับ CPU demo • Typhoon 8B ต้องรอผ่าน SLURM+Singularity (slide ถัดไป)

Lab A: Docker Ollama — ลองรัน LLM บน CPU

Step 1: คำนวณ port

```
MY_PORT=$((11400 + $(id -u) % 1000))
echo "My port: $MY_PORT"
```

Step 2: Run Ollama container

```
docker run -d --runtime=runc \
  --name ollama-$USER \
  -p ${MY_PORT}:11434 \
  -v $HOME/ollama_models:/root/.ollama \
  ollama/ollama
```

Step 3: Pull & Chat

```
docker exec -it ollama-$USER ollama pull tinyllama
docker exec -it ollama-$USER \
  ollama run tinyllama "What is Docker?"
```

Step 4: REST API

```
curl http://localhost:${MY_PORT}/api/chat -d '{
  "model": "tinyllama",
  "messages": [{"role":"user","content":"Hello"}],
  "stream": false
}'
```

SLURM + Ollama + Typhoon (GPU จริง)

sbatch ollama_gradio_run.sh → Ollama Serve + Gradio UI → cat port_forwarding_command → SSH Tunnel → Browser

ollama_gradio_run.sh (key sections)

```
#SBATCH -p defq  --gres=gpu:1g.18gb:1
#SBATCH --cpus-per-task=2  --mem=16G
#SBATCH --time=02:00:00

PORT_GRADIO=$(get_free_port)
OLLAMA_PORT=$(get_free_port)
SERVER=$(hostname)

# บันทึก SSH command ไว้ให้ copy
echo "ssh -L ${PORT_GRADIO}:${SERVER}:${PORT_GRADIO}\
  $USER@171.102.216.177" > port_forwarding_command

source activate deepseek
export OLLAMA_HOST=127.0.0.1:$OLLAMA_PORT
ollama serve > ollama_server.log 2>&1 &

# รอ Ollama พร้อม แล้วรัน Gradio
python ollama_web.py --host=0.0.0.0 \
  --port=$PORT_GRADIO \
  --ollama-url=http://127.0.0.1:$OLLAMA_PORT
```

หลัง sbatch — เช็ค job + SSH command

```
# ส่ง job
sbatch ollama_gradio_run.sh

# เช็ค status
squeue -u $USER

# อ่าน SSH command ที่ generate อัตโนมัติ
cat port_forwarding_command
```

Gradio UI — Pull Typhoon + Chat

```
# Local machine: รัน SSH tunnel
ssh -L PORT:rsu-training:PORT \
  test06@171.102.216.177

# Browser → http://localhost:PORT
# กด Pull: scb10x/llama3.1-
#   typhoon2-8b-instruct
# Chat: เลือกโมเดลแล้วคุย
```

✅ Ollama ติดตั้งใน userspace แล้ว • ไม่ต้องใช้ Singularity • Port random อัตโนมัติ — ไม่ชนกันระหว่าง user • SSH command generate ให้พร้อม copy-paste

Lab B: SLURM + Ollama + Typhoon — ขั้นตอนจริง

Step 1: CD repo + Activate env

```
cd ~/deepseek-with-ollama-on-supercomputer
conda activate deepseek
# ตรวจสอบ
python --version
ollama --version
```

Step 2: Submit SLURM job

```
sbatch ollama_gradio_run.sh
# ดู output ทันที:
tail -f slurm-<JOBID>.out
# รอ: ✅ Ollama ready
#     ✅ Gradio running
```

Step 3: Copy SSH tunnel command

```
# อ่าน command ที่ generate ให้:
cat port_forwarding_command

# เปิด Terminal ใหม่ บนเครื่องตัวเอง
# แล้ว paste command นั้น
```

Step 4: Browser → Pull → Chat TH

```
# Browser: http://localhost:<PORT>
# คลิก Pull Model:
#   scb10x/llama3.1-typhoon2-8b-instruct
# รอ download ~5 GB
# เลือก model → Chat ภาษาไทย!
```

💡 scancel <JOBID> เพื่อหยุด • Model (~5GB) เก็บใน ~/ollama/models — ใช้ได้ทุก job • conda env 'deepseek' เตรียมไว้พร้อมแล้ว ไม่ต้อง install เพิ่ม

สิ่งที่ได้เรียนรู้วันนี้

🧠 LLM Fundamentals

Token / Precision / Training vs Inference
ข้อจำกัดหลัก = GPU Memory

🔧 Ecosystem & Tools

HuggingFace / vLLM / TGI / Ollama / LoRA
เลือก tool ให้ตรงกับ use case

🐳 Container Basics

Docker CPU mode • SLURM+Singularity+MIG
Ollama + Typhoon2-8b ภาษาไทย

🚀 Workshop

Tech Stack Advisor: train → Dockerize
→ Singularity on HPC → HF Spaces